🧩 1. What is a Property in Python?

In Python, a **property** is a **special built-in feature** that lets you **control access to class attributes** —
without changing how they are accessed!

It helps make your code **cleaner, safer, and more Pythonic** 🐍

🔍 Normal Getter/Setter Way (Before Properties)

```
class Student:
    def __init__(self, name, grade):
        self.__name = name
        self.__grade = grade

    def get_grade(self):
        return self.__grade

    def set_grade(self, grade):
        if 0 <= grade <= 100:
            self.__grade = grade
        else:
            print("Invalid grade")

stu = Student("Alice", 90)
print(stu.get_grade())   # Access using method
stu.set_grade(95)        # Modify using method
```

That works fine, but looks **old-fashioned** and not very “Pythonic”.

⚡ **2. Using @property Decorator**

We can make this cleaner with the **@property decorator**.

```
class Student:
    def __init__(self, name, grade):
        self.__name = name
        self.__grade = grade

    @property
    def grade(self):
        return self.__grade   # acts like getter

    @grade.setter
    def grade(self, value):
        if 0 <= value <= 100:
            self.__grade = value
        else:
            print("Invalid grade value!")

stu = Student("Alice", 90)

print(stu.grade)    # instead of stu.get_grade()
stu.grade = 95      # instead of stu.set_grade(95)
print(stu.grade)
stu.grade = 120     # Invalid grade value!
```

🧠 Notice how we used **simple attribute-style access** (stu.grade) but it's actually **calling methods internally**.

That’s the power of @property — clean and controlled access.

🧱 3. Why Use Properties?

Without Property	With Property
obj.get_value()	obj.value
obj.set_value(50)	obj.value = 50
Verbose syntax	Cleaner syntax
Harder to refactor	Easier to maintain

✅ **Encapsulation with a clean interface**

🔒 4. Read-Only Property

If you only define a **getter** (no setter),
the property becomes **read-only**.

```
class Circle:
    def __init__(self, radius):
        self.__radius = radius

    @property
    def area(self):
        return 3.14 * (self.__radius ** 2)

circle = Circle(5)
print(circle.area)   # 78.5
circle.area = 100    # ❌ Error: can't set attribute
```

🧠 *area is calculated, not directly stored — it’s read-only.*

🧹 5. Deleter Property

You can even define what happens when someone deletes a property using @property_name.deleter.

```
class Person:
    def __init__(self, name):
        self.__name = name

    @property
    def name(self):
        return self.__name

    @name.deleter
    def name(self):
        print("Deleting name...")
        self.__name = None

p = Person("John")
print(p.name)
del p.name
print(p.name)
```

📜 **Output:**

```
John
Deleting name...
None
```

⚙️ 6. Full Example: Temperature Converter

Let's combine everything 👇

```
class Temperature:
    def __init__(self, celsius):
        self.__celsius = celsius

    @property
    def celsius(self):
        return self.__celsius

    @celsius.setter
    def celsius(self, value):
        if value < -273.15:
            print("Error: Temperature below absolute zero!")
        else:
            self.__celsius = value

    @property
    def fahrenheit(self):
        return (self.__celsius * 9/5) + 32

temp = Temperature(25)
print(temp.celsius)     # 25
print(temp.fahrenheit)  # 77.0
temp.celsius = -300     # Error: Temperature below absolute zero!
```

✅ *fahrenheit is a derived (read-only) property.*

🧪 7. Practice Exercise

Try this yourself 👇

```
# Exercise: Python Class Property

# 1. Create a class 'Rectangle'

#    - private attributes: __width, __height

# 2. Add properties:

#    - width and height with setter validation (should be > 0)

#    - area (read-only property = width * height)

# 3. Test by creating an object and updating values
```

📜 8. Summary Table

Decorator	Purpose	Example
@property	Getter	@property def price(self): ...
@<name>.setter	Setter	@price.setter def price(self, value): ...
@<name>.deleter	Deleter	@price.deleter def price(self): ...